CMOS Inverter Design and Simulation in Cadence Virtuoso Using GPDK090

Abstract

This report documents the complete workflow for designing and simulating a CMOS inverter in Cadence Virtuoso using the gpdk090 generic 90 nm technology library. The work was executed inside a VMware Linux environment and covers library and technology setup, schematic entry, symbol generation, creation of a dedicated test circuit, DC analysis for voltage transfer characteristics, transient analysis for dynamic behavior, and key observations about simulation parameters such as stop time. Supporting screenshots and handwritten notes are archived alongside the report for reproducibility

1. Introduction

The CMOS inverter is the fundamental building block of digital integrated circuits. Implementing an inverter in a custom IC design flow provides a practical introduction to device-level design, simulation, and verification in professional EDA tools. This project uses Cadence Virtuoso and the gpdk090 process design kit (PDK) to:

- Understand how to create and configure a design library attached to a technology file.
- Draw a transistor-level CMOS inverter schematic.
- Generate a symbol from the schematic and reuse it in a test circuit.
- Configure and run DC and transient simulations in ADE L.
- Interpret the resulting waveforms and voltage transfer characteristic.

All steps have been captured through screenshots and notes to enable others to follow the same path

2. Tools and Environment

2.1 Software and Hardware

- **Virtualization:** VMware virtual machine running Linux as the guest OS.
- **EDA Tool:** Cadence Virtuoso IC 6.1.7 with Spectre as the simulator.
- **PDK:** gpdk090 (generic 90 nm process).

Virtuoso and the PDK are launched from a terminal inside the VM using the `virtuoso &` command, which opens the CIW log window and associated design tools.

2.2 Project Artifacts

Two key documents are used to record the work:

- **Screenshot log:** captures the environment, command line, library manager, schematic editor, ADE windows, and final plots.
- **Handwritten notes:** define the project goal, to-do list, assumptions about technology and device sizing, and corrections to simulation parameters (such as changing transient stop time from 500 ms to 200 ns)

These artifacts are stored alongside this report in the repository for traceability.

3. Project Objective and Scope

The objective of the project is to design and verify a single-stage CMOS inverter using gpdk090 in Cadence Virtuoso. The scope includes:

- Creating a custom design library and attaching it to gpdk090.
- Implementing a transistor-level inverter schematic.
- Generating an inverter symbol.
- Building a **test circuit** that instantiates this symbol along with supply and pulse sources.
- Performing DC and transient simulations to validate inverter functionality.

Physical layout, DRC, and LVS are outside the scope of this particular iteration but can be added later as an extension.

4. Library and Technology Setup

4.1 Creating Library `Vansh_lib`

1. Launch Virtuoso from the terminal.
2. Open **Library Manager** via **Tools → Library Manager**.
3. Select **File → New → Library...**.
4. Enter `Vansh_lib` as the library name and choose a suitable directory path.
5. Confirm creation to generate an empty design library.

Screenshots in the project notes show the library creation dialog, confirming the path and library name.

4.2 Attaching the GPDK090 Technology Library

1. Immediately after creating the library, Virtuoso prompts to attach a technology file.
2. Choose **Attach to an existing technology library**.
3. Select `gpdk090` from the list of available tech libraries.
4. Confirm the attachment.

The CIW log reports that `Vansh_lib` has been successfully attached to `gpdk090`, meaning all devices and design rules from the PDK are now available inside the custom library.

5. Schematic Design of the CMOS Inverter

5.1 Creating the Inverter Cellview

1. In Library Manager, with **Library = Vansh_lib**, create a new cell:
   - **Cell Name:** `inverter`
   - **View:** `schematic`
2. Open the schematic editor for this cell.

5.2 Device Selection and Placement

Using **Create → Instance**, the following devices are placed from the gpdk090 libraries:

- One PMOS transistor.
- One NMOS transistor.

The PMOS is placed above and the NMOS below to follow the conventional CMOS inverter arrangement. The screenshot log shows the schematic editor with `pmos` and `nmos` instances placed, with their names and device parameters visible.

5.3 Connectivity and Pin Definition

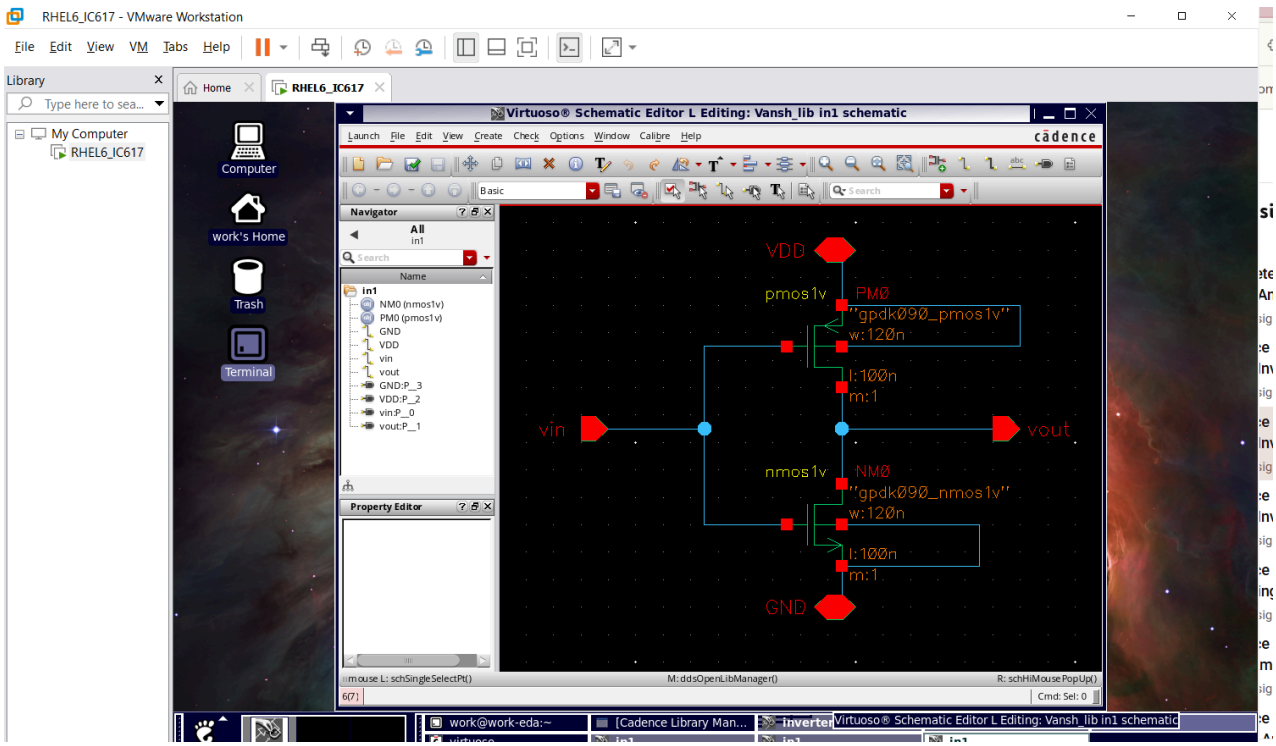
The inverter is wired as follows:

- **PMOS source** connected to the supply net `vdd`.
- **NMOS source** connected to the ground net `gnd`.
- **PMOS and NMOS drains** tied together to form the output node `vout`.
- **PMOS and NMOS gates** tied together to form the input node `vin`.

Pins are added using **Create → Pin** with the names:

- vin (input)
- vout (output)
- vdd (input)
- gnd (input)

A **Check and Save** operation confirms that there are no unconnected terminals or syntax errors.



5.4 Device Sizing

The exact sizing can be tuned depending on the learning goals. Notes for this project specify an NMOS length of approximately 100 nm and a width around 1 µm; the PMOS can be sized proportionally to balance switching characteristics For a first pass, any reasonable minimum-length sizes supported by the PDK are acceptable as long as the inverter behaves correctly in simulation.

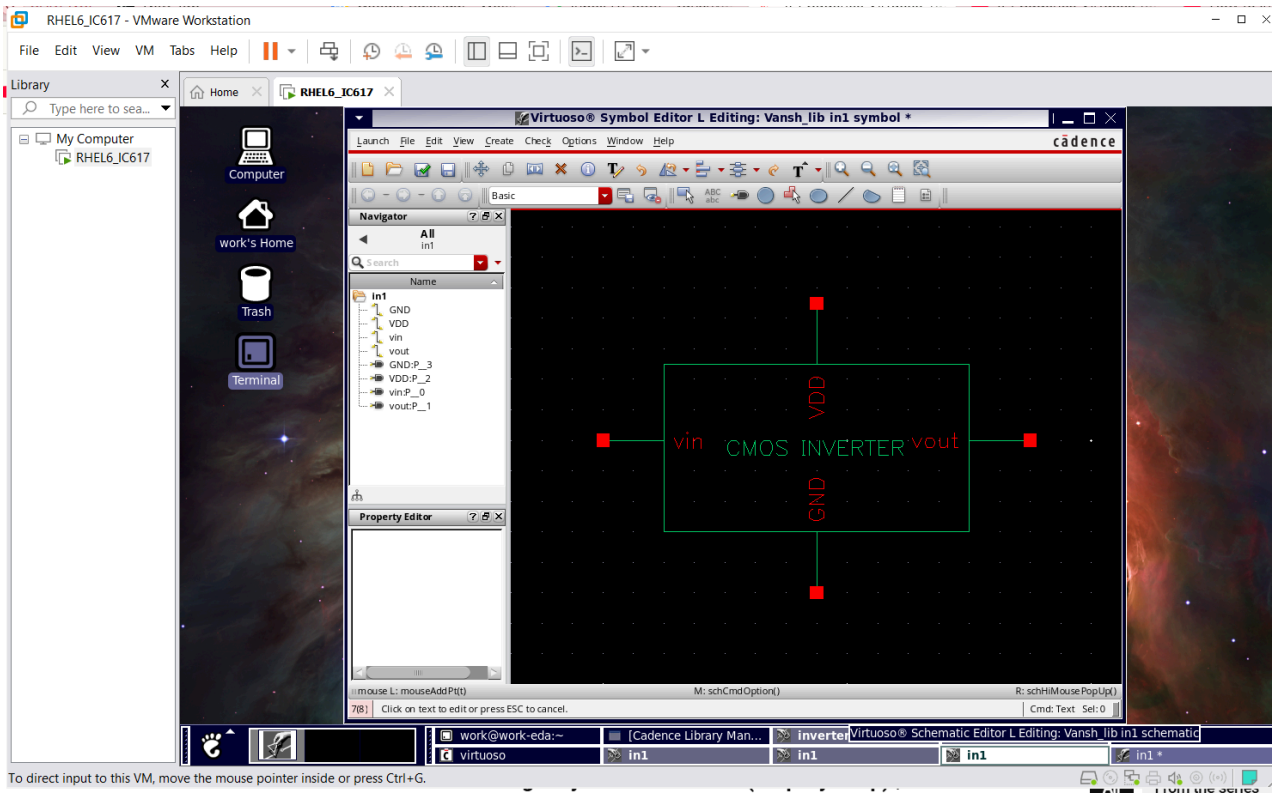
6. Symbol Generation

With the schematic complete and verified, a symbol is created:

1. In the schematic editor, use **Create → Cellview → From Cellview...**.
2. Choose source view schematic and target view symbol.
3. Virtuoso generates a default symbol frame with the pins vin, vout, vdd, and gnd mapped from the schematic.

4. The symbol can be edited to resemble a logic inverter symbol, with `vin` on the left, `vout` on the right, `vdd` at the top, and `gnd` at the bottom.
5. Save the symbol view.

This symbol will be instantiated in a separate **test circuit** used for simulation

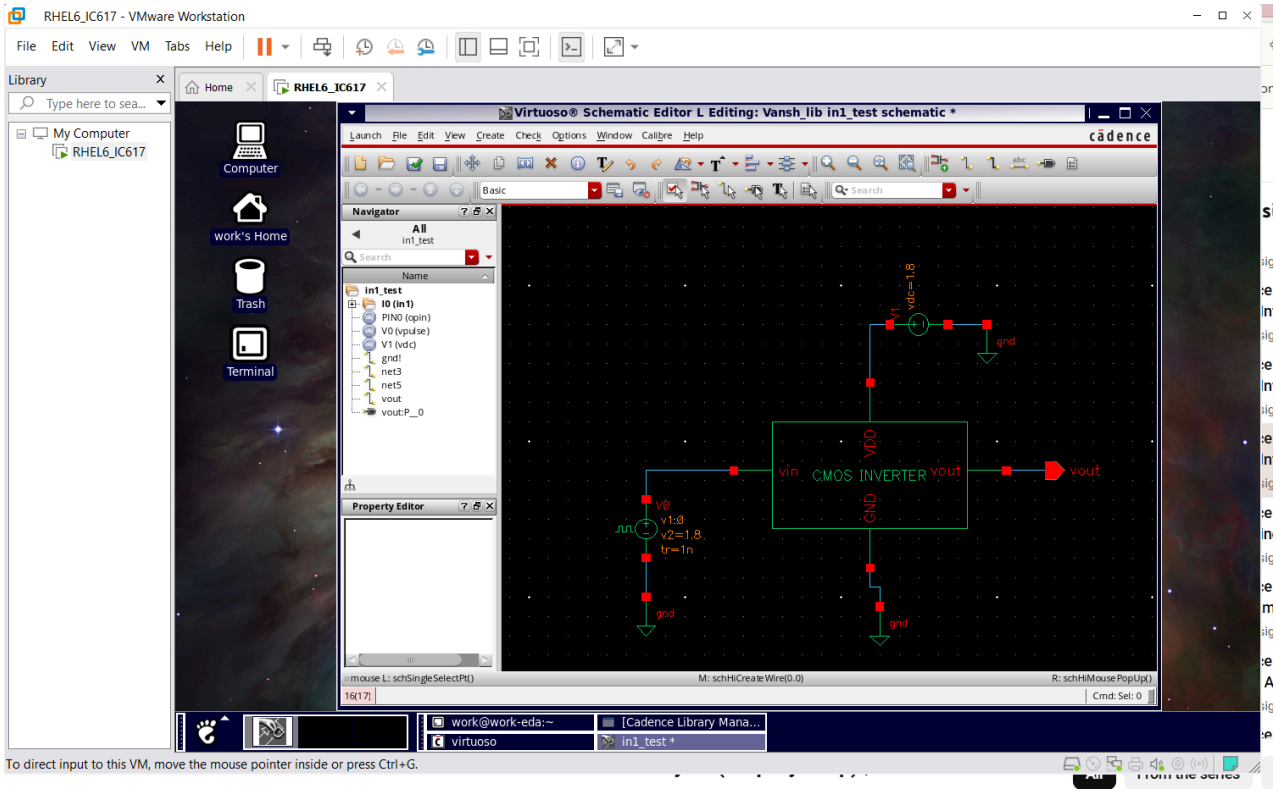


7. Test Circuit for the Inverter Symbol

7.1 Creating the Test Circuit Cell

To verify the inverter, a new cell is created that uses the inverter symbol along with sources and (optionally) a load capacitor:

1. In Library Manager, create a new cell:
   - **Library:** `Vansh_lib`
   - **Cell Name:** `inverter_test`
   - **View:** `schematic`
2. Open the schematic editor for `inverter_test`.



7.2 Instantiating the Inverter Symbol

Within inverter_test:

- Use **Create → Instance** to place the inverter symbol from Vansh_lib.
- Connect its pins as follows:
  - vdd → supply source
  - gnd → ground reference
  - vin → pulse input source
  - vout → observation node and optional capacitor load

7.3 Input and Supply Sources

The test circuit uses:

- A **DC voltage source** (vdc) to provide VDD = 1.8 V to the vdd node.
- A **pulse voltage source** (vpulse) as the input vin, configured with:
  - V1 = 0 V (logic 0)
  - V2 = 1.8 V (logic 1)

- Appropriate rise, fall, width, and period values to observe several switching cycles.

Notes from the project explicitly state the choice of `V1 = 0 V`, `V2 = 1.8 V`, and an initial transient stop time that was later corrected from 500 ms to 200 ns

7.4 Optional Output Load

For more realistic behavior, a small capacitor can be connected from `vout` to `gnd` to represent the load of subsequent logic gates. This is optional for basic functional verification but recommended for timing experiments.

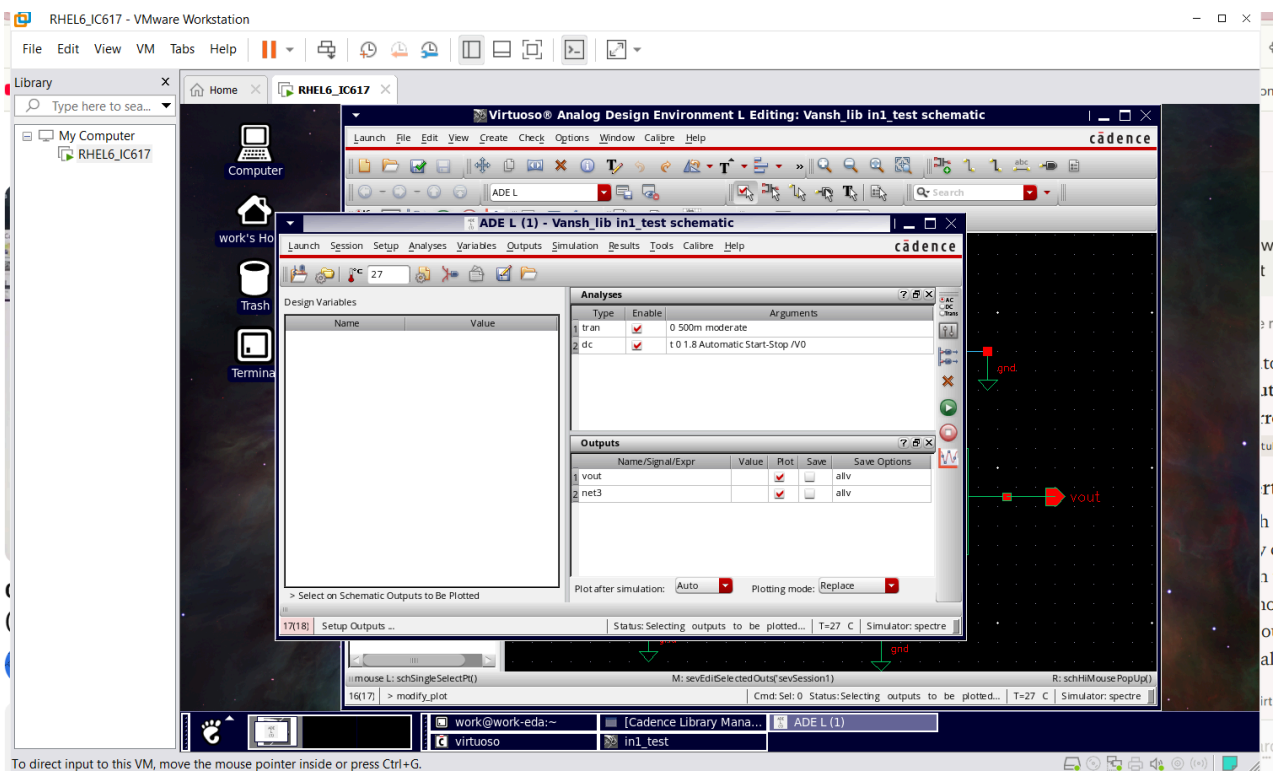
A **Check and Save** is run on the test circuit schematic before moving to simulation setup.

8. Simulation Setup in ADE L

All simulations are performed from the `inverter_test` schematic using ADE L with Spectre as the simulator.

8.1 Launching ADE L

1. With `inverter_test` open, select **Launch → ADE L**.
2. Confirm that the **Simulator** is set to **spectre**.
3. Add the signals `vin` and `vout` as outputs to be plotted.



8.2 Transient Analysis Configuration

The transient analysis evaluates the dynamic response of the inverter to the pulsed input.

- Analysis type: `tran` (transient).
- Correct stop time: **200 ns**.
- Initial time step and other tolerances can remain at defaults for this small circuit.

The notes highlight an important learning point: an initial choice of **500 ms** for the stop time is extremely large for a nanosecond-scale digital circuit, making simulation unnecessarily slow. This was recognized and corrected to **200 ns**, which is sufficient to observe multiple input transitions

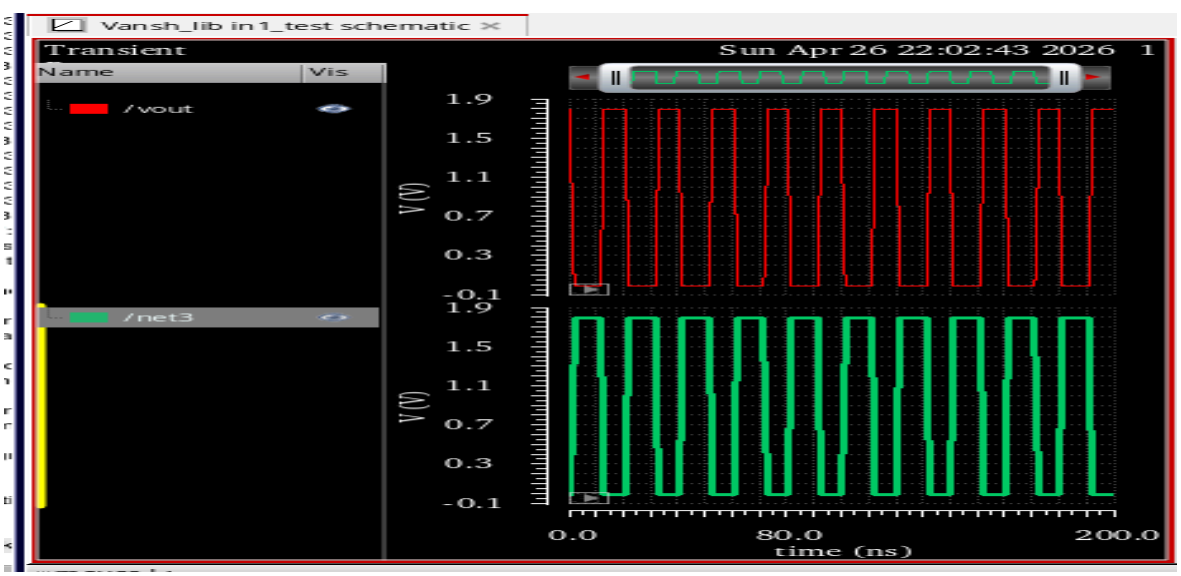
8.3 Running Transient Analysis

1. In ADE L, select **Netlist and Run**.
2. Once the simulation finishes, the **Virtuoso Visualization & Analysis** window opens.
3. Plot `vin` and `vout` versus time.

Expected behavior:

- When `vin` is low (≈ 0 V), `vout` remains high (≈ 1.8 V).
- When `vin` transitions high (≈ 1.8 V), `vout` is pulled low.
- When `vin` goes low again, `vout` returns high.

Screenshots in the project log show the transient plot with both input and output waveforms over a 0–200 ns window.



8.4 DC Analysis Configuration (Voltage Transfer Characteristic)

DC analysis is used to generate the voltage transfer characteristic (VTC) of the inverter.

- Analysis type: `dc`.
- Sweep variable: the DC input (e.g., the voltage source connected to `vin`).
- Sweep range: `0 V` to `1.8 V`.
- Step size: a small increment to capture the shape of the VTC (for example, a few millivolts per step).

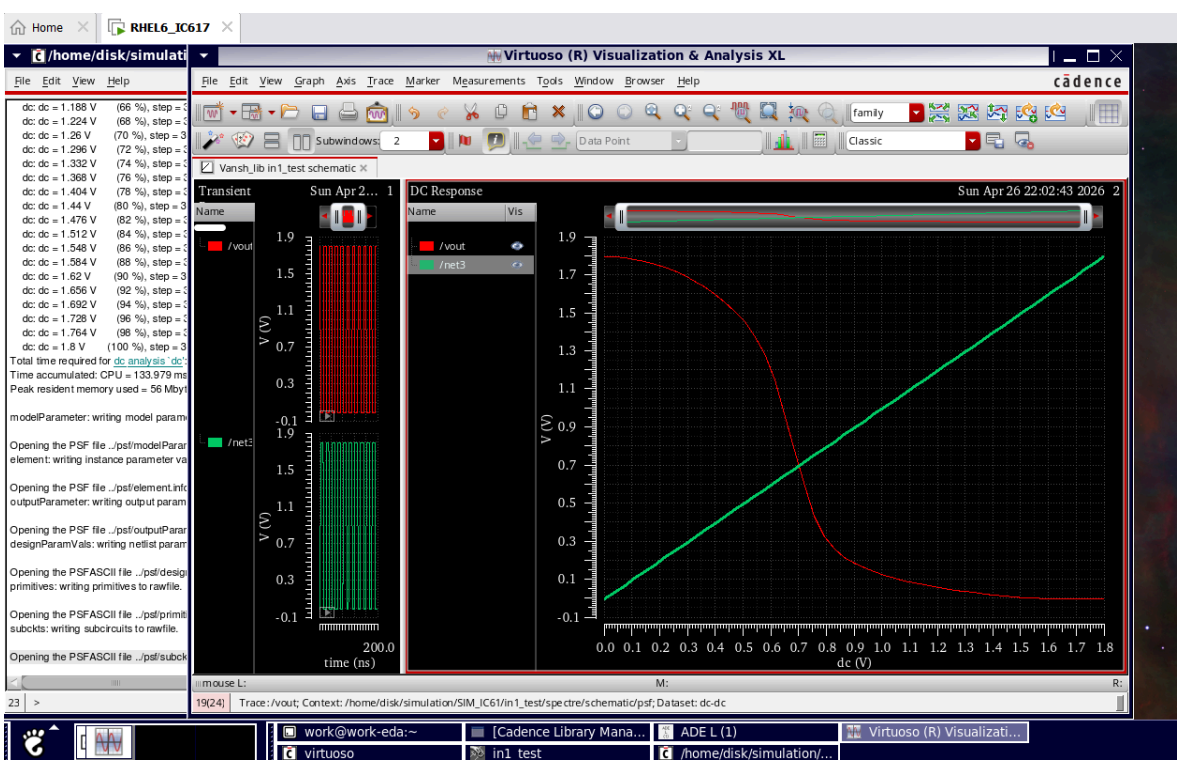
8.5 Running DC Analysis

1. Enable DC analysis in ADE L and run **Netlist and Run**.
2. In the waveform viewer, select the DC dataset and plot `vout` vs the swept input.

The plotted VTC shows:

- High output when input is near 0 V.
- A transition region where both transistors conduct and the output falls sharply.
- Low output when input approaches the supply voltage.

The DC logs demonstrate a sweep from 0 to 1.8 V and the associated computation of the inverter's operating point at each step.



9. Results and Discussion

9.1 Functional Verification

Both transient and DC analyses confirm that the designed circuit behaves as a valid CMOS inverter:

- In transient response, the output waveform is the logical inversion of the input pulse.
- In the DC characteristic, the output is high for low input and low for high input, with a sharp transition between the two states.

9.2 Switching Point

From the DC curve, the approximate switching point (where input and output cross in the transition region) appears near the mid-supply range (around 0.7–0.9 V for a 1.8 V supply). This switching threshold depends on the relative drive strengths and threshold voltages of the PMOS and NMOS devices, as well as their W/L ratios.

9.3 Impact of Stop Time Choice

A major practical lesson was the effect of transient stop time on simulation performance:

- **500 ms stop time:** Initially selected, this value is several orders of magnitude larger than necessary for a nanosecond-scale digital cell. Spectre must simulate a huge time window, resulting in very long runtimes.
- **200 ns stop time:** After recognizing the mismatch between device time constants and stop time, the value was changed to 200 ns. This is sufficient to cover several periods of the input pulse while keeping simulation time reasonable.

This experience underscores the importance of aligning simulation settings with the physical time scales of the circuit under test.

9.4 Limitations and Next Steps

The current project focuses on schematic-level design and simulation only. The following enhancements are natural next steps:

- Creating a physical layout for the inverter in Virtuoso Layout Suite.
- Running **DRC** (Design Rule Check) and **LVS** (Layout vs Schematic) on the layout.
- Performing post-layout extraction and re-running transient analysis with parasitics.
- Extending the design to NAND, NOR, or ring oscillator circuits using the inverter as a building block.

10. Conclusion

This project successfully demonstrates the full flow of designing and simulating a CMOS inverter in Cadence Virtuoso using the gpdk090 technology library. Starting from environment setup in a VMware Linux guest, a custom library `Vansh_lib` was created and attached to gpdk090. A transistor-level inverter schematic was drawn, converted to a symbol, and instantiated in a dedicated test circuit.

DC and transient simulations in ADE L confirmed correct inverter behavior. Along the way, the project highlighted the significance of appropriate simulation parameters, particularly transient stop time, and reinforced best practices in schematic organization and documentation. The supporting screenshots and notes provide a reproducible reference for future experiments and extensions.

The same methodology can now be applied to more complex digital logic modules and, with layout and verification, to complete custom cells suitable for integration in larger ASIC design flows.

Author: Vansh Vaghela

Project: CMOS Inverter – Cadence Virtuoso + gpdk090

Date: April 2026